

Received 15 May 2026, accepted 30 May 2026, date of publication 16 June 2026, date of current version 29 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3704367

RESEARCH ARTICLE

Scalable Multi-Agent Reinforcement Learning With Permutation-Free Networks

HYUNWOO PARK, BAEKRYUN SEONG, AND SANG-KI KO 

Department of Artificial Intelligence, University of Seoul, Seoul 02504, Republic of Korea

Corresponding author: Sang-Ki Ko (sangkiko@uos.ac.kr)

This work was supported by the National Research Foundation of Korea (NRF) grant funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant RS-2024-00456065.

ABSTRACT In cooperative multi-agent reinforcement learning (MARL), the permutation problem—where the state space grows exponentially with the number of agents—reduces sample efficiency. Additionally, many existing architectures struggle with scalability, as they rely on a fixed structure tied to a specific number of agents, limiting their applicability to environments with variable entity counts. While approaches such as graph neural networks (GNNs) and self-attention mechanisms have made progress in addressing these challenges, they have significant limitations: dense GNNs and self-attention mechanisms incur high computational costs. To overcome these limitations, we propose an architecture composed of a novel agent network and a scalable non-linear mixing network, both of which ensure permutation-freeness and scalability, enabling generalization to environments with variable numbers of agents. Our agent network significantly reduces the computational complexity of attention from $O(n^2d)$ to $O(nd)$, and our scalable hypernetwork enables efficient weight generation for non-linear mixing. Additionally, we introduce curriculum learning to improve training efficiency. Experiments on StarCraft Multi-Agent Challenge v2 (SMACv2) and Google Research Football (GRF) show that our proposed method generally outperforms representative MARL baselines, including HPN, UPDeT, VDN, and QMIX-based variants, particularly on SMACv2 scenarios with varying numbers and types of agents.

INDEX TERMS Multi-agent reinforcement learning (MARL), curriculum learning, permutation-free network, SMACv2, Google research football (GRF).

I. INTRODUCTION

Reinforcement learning (RL) has achieved remarkable success in various domains such as games [1], [2], [3], robotics [4], [5], and natural language processing [6], [7], [8]. However, traditional RL focuses on training a single agent, limiting its applicability to complex problems requiring interaction among multiple agents. To address this limitation, MARL has emerged as a framework that enables multiple agents to interact and learn toward a common goal. MARL offers significant potential to solve complex real-world challenges, such as dynamic resource allocation [9], traffic control [10], and team sports [11].

The associate editor coordinating the review of this manuscript and approving it for publication was Bing Li .

However, MARL faces significant challenges arising from the inherent complexity of multi-agent systems. One major hurdle is the scalability of learning algorithms to varying numbers of agents and environment entities, such as enemies, objects, and task-relevant features. Traditional MARL algorithms using multilayer perceptrons (MLPs) rely on fixed-size input representations [12], making them ill-suited for training with dynamic agent populations [13]. In addition, the exponential growth of the action space with the number of agents increases exploration difficulty [14], [15], requiring agents to spend prolonged periods exploring. Furthermore, the entities an agent observes are implicitly ordered in sequence, leading to permutation problems where the state space grows exponentially with the number of agents, thereby reducing sample efficiency [16].

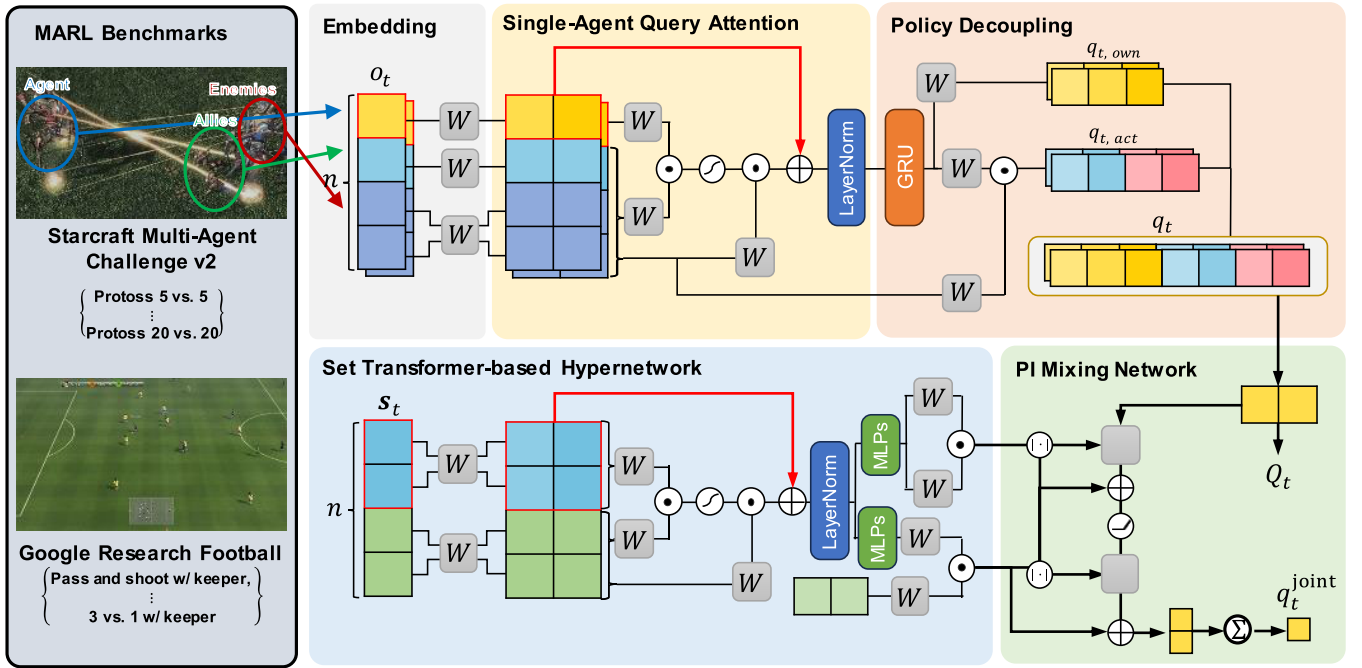


FIGURE 1. Overview of the proposed SPECTra framework. Gray boxes represent linear layers.

TABLE 1. Comparison of the proposed framework with baselines in terms of two key properties.

Algorithm	Scalable	Permutation-Free
SPECTra-VDN	✓	✓
HPN-VDN	✗	✓
UPDeT-VDN	✓	✓
SPECTra-QMIX ⁺	✓	✓
HPN-QMIX	✗	✗
UPDeT-QMIX	✗	✗

Recent transformer-based MARL methods, such as UPDeT [17], have attempted to improve scalability and generalization by processing variable-size observations and action configurations with attention mechanisms. However, these methods commonly rely on dense self-attention, whose computational cost grows quadratically with the number of observed entities. Moreover, existing transformer-based approaches primarily address permutation handling in the agent network, while the scalability and permutation-freeness of non-linear value mixing remain less explored [18].

To address these issues, we propose the **Scalable and Permutation-free Efficient Curriculum and Transferable** learning framework based on **Transformer** architecture, called the SPECTra framework. Figure 1 illustrates the overall architecture of the proposed framework. The agent network and mixing network of SPECTra enable learning independently of the number of agents, even with a fixed structure. The implicit ordering of observations and actions is addressed through permutation equivariance and

permutation invariance. As a result, our framework improves sample efficiency compared to traditional methods and can accommodate dynamic agent populations.

For the agent network, we propose single-agent query attention (SAQA), which reduces the time complexity to $O(nd)$. SAQA utilizes cross-attention by querying only information relevant to the observer agent. Unlike self-attention, which processes interactions among all observed entities in all directions, SAQA focuses only on entities with which the observer interacts, effectively improving scalability [19]. It also employs policy decoupling [17] to compute action values independently of the scalability and permutation problems. We also design a Set Transformer-Based Hypernetwork (ST-HyperNet) [12], [20] that generates weight matrices and bias vectors for the mixing network, ensuring scalability and permutation-freeness while maintaining the monotonicity constraint as in QMIX [12]. The mathematical properties of both the agent network and the mixing network further enable sample-efficient dynamic curriculum learning.

The main contributions of the paper are as follows:

- We propose the scalable and permutation-free MARL framework called SPECTra by employing Transformer-based architecture in agent networks and hypernetworks.
- We provide a theoretical analysis of the inference time of the proposed framework, especially permutation-free property and computational efficiency, and a qualitative analysis of the sample complexity of SAQA.
- We empirically demonstrate that SPECTra achieves competitive or superior performance compared with representative baselines such as UPDeT and HPN in the

evaluated SMACv2 and GRF settings, and further show that our framework is well-suited to curriculum learning and transfer learning.

II. RELATED WORK

A. MULTI-AGENT REINFORCEMENT LEARNING

Many cooperative MARL algorithms have used a centralized training decentralized execution (CTDE) [21], [22] framework to allow multiple agents to achieve a common goal. CTDE is a training paradigm that exploits global information, such as the joint state, joint actions, or other agents' observations, only during training to learn a centralized critic or centralized value function. This centralized information helps stabilize learning and address non-stationarity caused by simultaneously learning agents, while preserving decentralized policies that can be executed independently using only local observations. During execution, each agent selects its action without access to other agents' private information or the global state, making CTDE suitable for realistic cooperative settings with communication or observability constraints. Value Decomposition Network (VDN) [23] and QMIX [12] are representative value-based CTDE algorithms. VDN utilizes additive value decomposition to represent a linear joint-action value function as the sum of individual action values. The linear joint-action value function addresses some cooperation problems, such as the lazy agent problem, but a linear mixing network can represent only a limited class of centralized action-value functions. QMIX significantly improves the representational capability by redesigning the linear joint-action value function of VDN into a non-linear mixing network satisfying Individual-Global-Max (IGM) [24] using hypernetworks. In addition, QMIX with normalized optimization [25] has shown that QMIX can outperform its variants [26], [27], [28] on the SMAC [29] benchmark through regularized optimization. However, VDN and QMIX still cannot address the scalability and permutation problems [16], [18].

B. PERMUTATION-INVARIANT AND EQUIVARIANT MODELING FOR MULTI-AGENT SYSTEMS

Various MARL frameworks have utilized permutation equivariance and invariance to address observation and action permutation problems [30], [31], [32]. Among them, Hyper Policy Network (HPN) [18] has demonstrated effective solutions by independently embedding each observed entity using a permutation-equivariant hypernetwork, outperforming existing permutation-free architectures such as Deep Set [33], self-attention [34], and GNNs [35], [36], [37]. Since HPN has already compared against representative permutation-aware architectures, including Deep Set-, self-attention-, and GNN-based models, we do not repeat these comparisons as primary baselines. Instead, we focus on stronger and more directly relevant baselines, namely HPN, UPDeT, VDN, and QMIX-based variants, to evaluate scalability and permutation-freeness under value-decomposition-based CTDE settings.

However, HPN inherently applies a summation operation to achieve permutation invariance over independent entity

embeddings, preventing it from selectively focusing on specific entities when selecting the optimal action. Additionally, integrating a non-linear mixing network such as QMIX cannot guarantee permutation invariance due to its reliance on simple MLPs, which are not inherently permutation-invariant. Furthermore, both the agent network and the mixing network face challenges in transfer learning and dynamic curriculum learning, as their architectures depend on the number of agents. In contrast, our proposed SPECTra framework uses SAQA to ensure computationally efficient permutation equivariance in the agent network and employs a permutation-invariant mixing network via a hypernetwork, making it completely independent of permutation problems.

C. DYNAMIC CURRICULUM LEARNING

Curriculum learning has emerged as a powerful paradigm for accelerating MARL. By adopting a staged learning approach, it begins with simpler tasks and progressively transitions to more complex challenges. In MARL, one standard method for defining task difficulty involves varying the number of agents. Increasing the number of agents inherently introduces greater environmental dynamics and stochasticity, making coordination among agents significantly more challenging [38]. For instance, Dyma-CL [13], a GNN-based curriculum learning framework, applied curriculum learning to simple agent networks. Their findings demonstrate that even simple algorithms such as IQL [39] and VDN can achieve more efficient training by gradually increasing the number of agents instead of training from scratch. Recent advancements in automatic curriculum learning [40], [41], [42] have introduced frameworks that automatically generate and adapt curricula by dynamically assigning the number of agents. These developments underscore the potential of designing models capable of adapting to varying agent counts, motivating further exploration into architectures that remain invariant to the number of agents.

A notable related effort is the UPDeT [17] framework, which, while not explicitly incorporating curriculum learning, offers an important example of a universal MARL pipeline. UPDeT employs transformer-based self-attention mechanisms to handle observations and action configurations of dynamic sizes using a unified architecture. This design partially enables zero-shot learning across agent configurations. However, UPDeT's reliance on dense self-attention connections imposes an $O(n^2d)$ computational complexity [34], and its MLP-based hypernetwork limits the scalability of non-linear mixing networks because direct MLP-based parameter generation can become difficult to scale for high-dimensional target mixing networks [20]. Moreover, self-attention computations significantly increase sample complexity due to dense connections, diminishing the benefits of scalability [19]. In contrast, our proposed SPECTra framework utilizes SAQA to drastically reduce UPDeT's $O(n^2d)$ computational complexity to $O(nd)$ while maintaining permutation equivariance. Furthermore, while UPDeT primarily focused on solving the permutation problem in the agent network, we effectively address the

TABLE 2. Summary of key notations used in this paper.

Symbol	Description
n	Number of entities (agents and enemies).
n_h	Number of attention heads
d	Dimension of a feature vector
d_o	Dimension of each observation $o_{i \rightarrow j}$
d_h	Dimension of each hidden vector
d_k	Dimension of each attention head, such that $n_h \times d_k = d_h$
S	Set of global states of the environment
A	Set of entities
$A_{\text{agent}} \subset A$	Set of agents
$a_i \in A_{\text{agent}}$	Agent i
$m = A_{\text{agent}} $	Number of agents
U	Set of actions
$U_i \subset U$	Set of actions of agent i
$u_i = (\mathbf{u})_i \in U_i$	Action of agent i
O	Set of <i>entity-wise observation</i>
$o_{i \rightarrow j} = (\mathbf{o}_i)_j \in O$	Entity-wise observation from entity a_i to a_j
$e_{i \rightarrow j}$	Embedded observation from agent a_i to entity a_j
$q_{i,t,\text{own}}$	Action values of non-interactive actions from agent i
$q_{i,t,\text{act}}$	Action values of interactive actions from agent i
q_t^{joint}	Composed joint action value

permutation problem in the mixing network as well by using ST-HyperNet.

III. BACKGROUND

A. PROBLEM FORMULATION

We formulate the MARL problem as a decentralized partially observable Markov decision process (Dec-POMDP) [43], [44] with an 8-tuple $M = \langle S, A, U, P, r, O, n, \gamma \rangle$. Let S be the global state of the environment, $A = \{a_1, \dots, a_n\}$ be the set of n entities, and U be the action space. Since an agent is an independent decision-making entity that interacts with the environment, we denote the set of agents by A_{agent} and the number of agents by m . Namely, $|A_{\text{agent}}| = m$, where $n > m$.

We denote the *entity-wise observation* from entity a_1 to entity a_2 by $o_{1 \rightarrow 2} \in O$. Then, the local observation of entity a_i is denoted by the observation vector $\mathbf{o}_i$, where $o_{ij} = o_{i \rightarrow j}$. An agent a_i selects an action $u \in U$ at each time step t , and actions selected by m agents form a joint action $\mathbf{u} \in U^m$. We suppose that each agent a_i has its own action space U_i , which is independent of the permutation of agents. We also let $P(s'|s, \mathbf{u}) : S \times U^m \times S \rightarrow [0, 1]$ denote the state transition probability from s to s' given the joint action $\mathbf{u}$. The immediate reward function is defined as $r(s, \mathbf{u}) : S \times U^m \rightarrow \mathbb{R}$, and $\gamma \in [0, 1]$ is a discount factor. At time step t , an agent a_i observes a joint observation $\mathbf{o}_{i,t}$ and takes an action $u_{i,t} \in U_i$ using the joint policy $\pi_\theta(\mathbf{u}_t | \{\mathbf{o}_{i,t}\}_{i=1}^m)$ parameterized by θ .

We define the joint action value at time step t following the policy π_θ as follows:

$$Q^{\pi_\theta}(s_t, \mathbf{u}_t) = \mathbb{E}[R_t | s_t, \mathbf{u}_t],$$

where $R_t = \sum_{\alpha=0}^{\infty} \gamma^\alpha r_{t+\alpha}$ is the discounted return. The key notations used throughout this paper are summarized in Table 2.

Algorithm 1 Training Procedure of SPECTra

- 1: Initialize SPECTra networks, target networks, and replay buffer $\mathcal{D}$.
- 2: **for** each episode **do**
- 3: Reset the environment and obtain initial observations and global state.
- 4: **for** each time step t **do**
- 5: **for** each agent i **do**
- 6: Compute agent representation using SAQA.
- 7: Estimate masked individual action values $q_{i,t}^{\text{mask}}$ via permutation-free policy decoupling.
- 8: Select action $u_{i,t}$ using an exploration policy.
- 9: **end for**
- 10: Execute joint action $\mathbf{u}_t$ and store the transition in $\mathcal{D}$.
- 11: Sample a mini-batch from $\mathcal{D}$.
- 12: Generate mixing weights and biases using ST-HyperNet.
- 13: Compute joint action values and TD targets.
- 14: Update SPECTra by minimizing the TD loss and periodically update target networks.
- 15: **end for**
- 16: **end for**

B. PERMUTATION INVARIANCE / PERMUTATION EQUIVARIANCE

In MARL, the order of agents can change, leading to an explosion in the number of possible input representations due to permutations. In the worst case, a naive approach would require considering all $n!$ possible permutations. Moreover, the output complexity also increases, as the joint action space scales as U^m .

Let us first formally define the PI and PE attributes from the perspective of MARL as follows:

Definition 1 (Permutation): Let S_n be the set of all bijective functions (permutations) from the set $\{1, 2, \dots, n\}$ to itself.

Definition 2 (Permutation Equivariance): A function $f: X^n \rightarrow Y^n$ is permutation equivariant iff for any permutation $\sigma \in S_n$ and an input matrix $X \in \mathbb{R}^{n \times d}$, $f(P_\sigma X) = P_\sigma f(X)$, where $P_\sigma \in \mathbb{R}^{n \times n}$ is the permutation matrix corresponding to σ , which acts on X by permuting its rows.

Definition 3 (Permutation Invariance): A function $f: X^n \rightarrow Y^n$ is permutation invariant iff for any permutation $\sigma \in S_n$ and an input matrix $X \in \mathbb{R}^{n \times d}$, $f(P_\sigma X) = f(X)$, where $P_\sigma \in \mathbb{R}^{n \times n}$ is the permutation matrix corresponding to σ .

To ensure that the overall framework remains permutation-free, it is not sufficient for the model architecture alone to be permutation-invariant or permutation-equivariant. The structure of observations and states must also meet specific conditions as follows:

- 1) First, each agent has a fixed index for encoding its own observation, meaning that the way an agent perceives itself does not change regardless of how other agents are ordered.

- 2) Second, when choosing an action, an agent can either take an action that directly affects itself (e.g., move) or an action that interacts with another entity (e.g., attack).
- 3) Lastly, since observations are represented as decomposable vectors, entities that are not directly observable due to partial observability should be masked, ensuring that missing information does not introduce unintended dependencies.

Under these structural constraints, the entire learning process remains permutation-free, leading to more stable and efficient training.

IV. METHODOLOGY

In this section, we describe the agent network and mixing network of SPECTra in detail. First, we explain the important elements of the agent network: SAQA and action value estimation via policy decoupling. Then, we prove that the agent network is scalable and permutation-equivariant. We also use a Set Transformer [45]-based hypernetwork called ST-HyperNet to show that non-linear mixing networks can estimate joint-action values in a permutation-invariant way and prove that the mixing network is scalable and permutation-invariant.

Algorithm 1 summarizes the overall training procedure of SPECTra. In the following subsections, we describe its three main components: SAQA-based agent representation, permutation-free policy decoupling, and ST-HyperNet-based value mixing.

SPECTra's scalability also enables a simple curriculum strategy in which parameters learned in smaller scenarios are reused in larger scenarios. We describe the detailed curriculum protocol in Section V-B.

A. LIGHTWEIGHT ATTENTION MECHANISM: SINGLE-AGENT QUERY ATTENTION (SAQA)

To address the high computational cost of the self-attention mechanism, there have been alternative attention mechanisms proposed to reduce its complexity from $O(n^2d)$ to $O(nd^2)$, such as Efficient Attention [46] and Hydra-Attention [47]. However, these algorithms have limited applicability because they require the hidden dimension d to be smaller than the number n of agents to be efficient. These limitations highlight the need for more tailored approaches to efficient attention mechanisms in MARL, ensuring computational scalability while preserving critical agent-centric information.

On the other hand, we propose a lightweight attention mechanism called *single-agent query attention (SAQA)*, which performs attention operations with the observer agent's information as a query and contextualizes the agent's embedding using relevant observations using the attention mechanism. In this way, we can achieve the computational cost of $O(nd)$ instead of $O(n^2d)$. Formally, self-attention computes pairwise attention scores for all n observed entities, requiring an $n \times n$ attention matrix and resulting in $O(n^2d)$ complexity. In contrast, SAQA uses only the observer agent as a single query and attends to the n entity embeddings once, requiring a $1 \times n$ attention score vector.

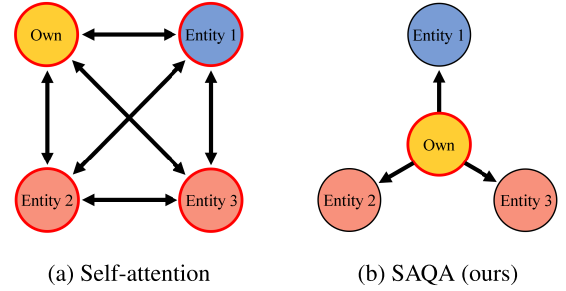


FIGURE 2. Comparison of connection density between self-attention and single-agent query attention.

Therefore, the attention computation scales linearly with the number of observed entities, yielding $O(nd)$ complexity. Figure 2 indicates the difference in structural complexity between self-attention and SAQA.

Given the observation vector $\mathbf{o}_i$ from agent a_i , such that $\mathbf{o}_{ij} = \mathbf{o}_{i \rightarrow j} \wedge j \in A_i^{\text{obs}} \forall j$, we first embed the observations via three separate linear layers E_{own} , E_{ally} , and E_{enemy} as follows:

$$\begin{aligned} e_i &= \{E_{\text{own}}(\mathbf{o}_{i \rightarrow i})\} \cup \{E_{\text{ally}}(\mathbf{o}_{i \rightarrow j}) \mid j \in A_{\text{ally}}\} \\ &\quad \cup \{E_{\text{enemy}}(\mathbf{o}_{i \rightarrow j}) \mid j \in A_{\text{enemy}}\} \subset \mathbb{R}^{d_h}. \end{aligned}$$

Note that $E_*(\mathbf{o}_{i \rightarrow j}) \in \mathbb{R}^{d_h}$ is a vector that encodes the observation of entity a_j from agent a_i and defined as $\mathbf{o}_{i \rightarrow j} \mathbf{W}_*$, where $\mathbf{W}_* \in \mathbb{R}^{d_o \times d_h}$ for $*$ in $\{\text{own}, \text{ally}, \text{enemy}\}$. Note that ' $*$ ' is determined by the type of entity a_j from a_i 's perspective (e.g., $*$ = own if $i = j$). Let us denote $E_*(\mathbf{o}_{i \rightarrow j})$ by $e_{i \rightarrow j}$ for simplicity.

Let n_h be the number of attention heads in each layer. Then, $n_h \times d_k = d_h$, where d_k is the hidden dimension of each attention head. We compute the query, key, and value vector for the k -th attention head by applying a linear transformation from d_h to d_k as follows:

$$Q_{i \rightarrow j}^{(k)}, K_{i \rightarrow j}^{(k)}, V_{i \rightarrow j}^{(k)} = e_{i \rightarrow j}^\top \mathbf{W}_Q^{(k)}, e_{i \rightarrow j}^\top \mathbf{W}_K^{(k)}, e_{i \rightarrow j}^\top \mathbf{W}_V^{(k)}.$$

Now, we compute the attention-weighted vector for agent i :

$$e_{i, \text{att}}^{(k)} = \left[\text{softmax} \left(\frac{Q_{i \rightarrow i}^{(k)} (K_i^{(k)})^\top}{\sqrt{d_k}} \right) V_i^{(k)} \right]^\top \in \mathbb{R}^{d_k}.$$

We concatenate the vectors from n_h heads as follows:

$$e_{i, \text{att}} = \left[(e_{i, \text{att}}^{(1)})^\top \dots (e_{i, \text{att}}^{(d_h)})^\top \right]^\top \in \mathbb{R}^{d_h}$$

Lastly, we apply layer normalization after combining the attention-weighted vector with the input query embedding via a residual connection:

$$e'_i = \text{LayerNorm}(e_{i \rightarrow i} + e_{i, \text{att}}) \in \mathbb{R}^{d_h}.$$

We can prove that the entire attention computation is permutation-invariant. The key reason is that SAQA uses the observer agent's own embedding as a fixed query and applies the same key/value transformations to each observed entity.

Therefore, permuting the order of observed entities only permutes the corresponding key/value pairs, while the softmax-weighted summation over these entities remains unchanged. This makes the contextualized observer embedding invariant to the input entity ordering.

Proposition 1: The single-agent query attention of the SPECTra framework is a permutation-invariant function.

Proof: Let P_σ be a permutation matrix applied to the entity embeddings. Since the same key and value projections are shared across entities, applying P_σ only permutes the rows of the key and value matrices, i.e., $\mathbf{K}_i$ and $\mathbf{V}_i$. The query $Q_{i \rightarrow i}$ is computed from the observer agent itself and is not permuted with the other observed entities. Thus, the attention scores are permuted in the same way as the value vectors. Because the weighted sum aggregates over all entities, the final attention output remains unchanged under the permutation. Therefore, SAQA is permutation-invariant with respect to the ordering of observed entities. $\square$

B. PERMUTATION-FREE POLICY DECOUPLING

Note that actions targeting other entities should be determined in a permutation-equivariant manner, as the input order should not affect the optimal decision-making of the current agent. We design the policy network of each agent to be permutation-equivariant by utilizing the permutation-invariant contextualized embedding e'_i from SAQA and the individual entity observations that are also free from the permutation problem.

We improve the policy decoupling proposed by UPDeT [17] to output action values permutation-equivariant using linear layers and inner products. Our policy decoupling uses the attention-based embedding of the current agent and the embedding of each observation-entity to compute action values for entities and optimize the policy at an action-group level.

To compute action values based on the action-observation history of agent a_i , we employ the Gated Recurrent Unit (GRU) cell as follows:

$$h_{i,t} = \text{GRUCell}(e'_i, h_{i,t-1}) \in \mathbb{R}^d.$$

From the hidden state $h_{i,t}$, we compute two features: one ($q_{i,t,\text{own}}$) for computing the action value for the actions on the agent itself (a_i) and the other ($Q_{i,t,\text{act}}$) for the actions on other agents (including enemies and allies) as follows:

$$q_{i,t,\text{own}} = \left[h_{i,t}^\top \mathbf{W}_{\text{own}}^Q \right]^\top \in \mathbb{R}^{|U_{\text{own}}|}$$

$$Q_{i,t,\text{act}}^{(k)} = e_i^\top \mathbf{W}_{\text{act}}^{(k),Q} \in \mathbb{R}^{1 \times d_k} \text{ for } 1 \leq k \leq n_h.$$

Now we apply a linear transformation to n embedded observations as follows:

$$\mathbf{K}_{i,t,\text{act}}^{(k)} = (e_{i \rightarrow i} \ e_{i \rightarrow 1} \ \dots \ e_{i \rightarrow i-1} \ e_{i \rightarrow i+1} \ \dots \ e_{i \rightarrow n})^\top$$

$$\times \mathbf{W}_{\text{act}}^{(k),K} \in \mathbb{R}^{n \times d_k}.$$

Now we are ready to compute the q -values for the actions that can be performed on the other agents by conducting the

dot product operations between a Q -vector and K -matrix:

$$q_{i,t,\text{act}} = \left[\sum_{k=1}^{d_h} \frac{Q_{i,t,\text{act}}^{(k)} \left(\mathbf{K}_{i,t,\text{act}}^{(k)} \right)^\top}{d_h \sqrt{d_k}} \right]^\top \in \mathbb{R}^n.$$

Note that the q -vector is the result of the scaled inner product of $Q_{i,t,\text{act}}^{(k)}$ and $\mathbf{K}_{i,t,\text{act}}^{(k)}$. The K -matrix represents the latent information of individual observations of all agents. By applying the inner product, the observer agent can estimate the value of taking action on the other agents based on its observations.

Finally, we have the action value of a_i at t as follows:

$$q_{i,t} = (q_{i,t,\text{own}}^\top \ q_{i,t,\text{act}}^\top)^\top \in \mathbb{R}^{|U|}$$

$$q_{i,t}^{\text{mask}} = \mathbf{Mask}_{i,t,\text{act}} \odot q_{i,t} + (1 - \mathbf{Mask}_{i,t,\text{act}}) \odot (-\infty),$$

$\mathbf{Mask}_{i,t,\text{act}} \in \{0, 1\}^{|U|}$ is a mask that filters out non-observable entities from actions. Note that $|U_{\text{own}}| + n = |U|$, since we split the action space into two categories: $q_{i,t,\text{own}}$ computes the action values for the agent's own actions, and $q_{i,t,\text{act}}$ computes the action values for actions targeting other entities. We can use the inner product to output $q_{i,t,\text{act}}$ that is permutation-equivariant to the permutations of allies and enemies, and $\mathbf{W}_{\text{own}}^Q$ to make permutation-independent $q_{i,t,\text{own}}$ permutation-invariant.

Proposition 2: The policy decoupling of SPECTra framework is a permutation-equivariant function.

Proof: By Proposition 1, the contextualized observer embedding e'_i is invariant to the ordering of observed entities. The action values for self-directed actions, $q_{i,t,\text{own}}$, are computed only from the hidden state of the observer agent and are therefore unaffected by permutations of other entities. For entity-interaction actions, the key matrix $\mathbf{K}_{i,t,\text{act}}$ is computed by applying the same linear transformation to each entity embedding. Thus, if the order of observed entities is permuted, the rows of $\mathbf{K}_{i,t,\text{act}}$ are permuted in the same way. Since $q_{i,t,\text{act}}$ is computed by inner products between the observer-dependent query and these entity-wise keys, the resulting action values are permuted according to the same entity permutation. Therefore, the output action values for entity-interaction actions are permutation-equivariant, while self-directed actions remain permutation-invariant. Combining these two parts yields a permutation-equivariant policy decoupling module. $\square$

C. SET TRANSFORMER-BASED HYPERNETWORK

Hypernetworks [20] refer to a class of neural architectures where one neural network generates the weights of another network. To ensure the permutation-equivariance of the mixing network, we employ the Set Transformer [45] that consists of the permutation-equivariant encoder and the permutation-invariant decoder.

First we compute the query $Q_{i,t}^{W,(k)}$ and key $\mathbf{K}_{i,t}^{W,(k)}$ values for the agent a_i by applying a linear transformation to the state representation of a_i . Then, STHyperNet computes the

weight matrix and the bias vector of the mixing network as follows:

$$W_t^1 = \sum_{k=1}^{n_h} \text{abs} \left(\frac{Q_{i,t}^{W,(k)} (\mathbf{K}_{i,t}^{W,(k)})^\top}{d_h \sqrt{d^{\text{mix}}/d_h}} \right) \in \mathbb{R}_+^{m \times m}$$

$$b_t^1 = \sum_{k=1}^{n_h} \frac{Q_{i,t}^{b,(k)} (\mathbf{K}_{i,t}^{b,(k)})^\top}{d_h \sqrt{d^{\text{mix}}}} \in \mathbb{R}^m.$$

$\text{abs}(\cdot)$ is the element-wise absolute value function.

As in QMIX [12], absolute value operations are applied to the weights W_t^1 of the mixing network to ensure monotonic value decomposition when computing the joint value function. Additionally, inspired by the decoder of the Set Transformer, we generate the permutation-equivariant bias vector b_t^1 using a random seed vector.

Similarly, we compose a hypernetwork for the joint Q-function with random seed vectors $q_t^{(k)} \in \mathbb{R}^{d_k}$.

$$W_t^2 = \sum_{k=1}^{n_h} \text{abs} \left(\frac{(q_t^{W,(k)})^\top (\mathbf{K}_{i,t}^{W,(k)})^\top}{d_h \sqrt{d^{\text{mix}}/d_h}} \right)^\top \in \mathbb{R}_+^m$$

$$b_t^2 = \sum_{l=1}^m \left[\sum_{k=1}^{n_h} \frac{Q_t^{b,(k)} (\mathbf{K}_{i,t}^{b,(k)})^\top}{d_h \sqrt{d^{\text{mix}}}} \right]_l \in \mathbb{R}$$

Proposition 3: ST-HyperNet of the SPECTra framework is a permutation-equivariant function.

Proof: ST-HyperNet applies shared linear projections to the state representations of agents and then computes attention-based interactions using the same parameters for all agents. Therefore, when the input agent order is permuted by P_σ , the corresponding query, key, and value representations are permuted in the same way. The generated weight matrix and bias vector consequently follow the same permutation structure as the input agent ordering. Thus, ST-HyperNet is permutation-equivariant with respect to permutations of agent ordering. $\square$

D. PERMUTATION-INVARIANT MIXING NETWORK

Since $W_t^1, b_t^1, W_t^2, b_t^2$ obtained using ST-HyperNet are permutation-equivariant, we can compute a permutation-invariant joint-action value using ST-HyperNet as follows:

$$W_t^1, b_t^1 = \text{ST-HyperNet}^1(S_t)$$

$$W_t^2, b_t^2 = \text{ST-HyperNet}^2(S_t)$$

$$Q_t = [(q_{1,t}^{\text{mask}})^\top \dots (q_{m,t}^{\text{mask}})^\top]^\top \in \mathbb{R}^m$$

$$H_t^{\text{mix}} = \text{ReLU}(Q_t^\top W_t^1 + (b_t^1)^\top) \in \mathbb{R}^{1 \times m}$$

$$q_t^{\text{joint}} = H_t^{\text{mix}} W_t^2 + b_t^2 \in \mathbb{R}$$

Intuitively, since the weights and biases generated by the two ST-HyperNet modules depend on the order of the agents provided as input, the sequence of q -values follows the same order as the weights and biases, ensuring that H_t^{mix} remains

permutation-equivariant. Finally, the matrix multiplication and vector addition between H_t^{mix} and the weight matrix and bias vector, which share the same permutation, are guaranteed to be permutation-invariant.

Proposition 4: The mixing network of SPECTra framework is a permutation-invariant function.

Proof: Let P_σ denote a permutation of the agent ordering. Under this permutation, the vector of individual action values is transformed as $Q_t \mapsto P_\sigma Q_t$. By Proposition 3, the weights and biases generated by ST-HyperNet are transformed consistently with the same permutation. Therefore, the intermediate hidden representation H_t^{mix} is permuted equivariantly. In the final mixing step, the permuted hidden representation is multiplied by the correspondingly permuted weight vector and combined with the invariant scalar bias. Since both the input values and the generated mixing parameters share the same permutation, the final scalar joint-action value remains unchanged. Hence, the mixing network is permutation-invariant. $\square$

V. EXPERIMENT

We evaluate the proposed SPECTra agent network and mixing network on SMACv2 and GRF. We first evaluate our contributions on SMACv2, a successor to SMAC that addresses the limitations of SMAC's limited stochasticity. SMACv2 introduces probabilistic unit sampling and diverse initial state distributions, incorporating random team compositions, starting positions, and unit types. This increased stochasticity highlights the necessity of addressing the permutation problem, reinforcing our motivation.

To ensure that SMACv2's training difficulty increases with the number of agents, we turn off the SMACv2 action mask option to reduce the frequency of masked actions. Additionally, to account for the decentralized execution of CTDE, we set the probability of an agent observing an enemy within its sight range to $p = 1.0$. Under these settings, SMACv2 significantly increases training difficulty as the number of agents grows. To smooth the variability of the experimental results and clarify the trend, we applied an exponentially weighted moving average (EWMA) to the experimental results, with a smoothing factor of 0.99. All experiments were conducted with 10 random seeds, and the reported learning curves show the average performance across seeds. For a fair comparison, all methods were evaluated under the same environment configurations, training budgets, evaluation intervals, and random seed settings. For baseline methods, we followed the hyperparameter settings reported in the original papers or available implementations whenever possible; when environment-specific settings were unavailable, we used comparable settings under the same experimental protocol. Because MARL performance can vary substantially across random seeds, we avoid making claims based on a single run and instead report trends based on 10-seed averages together with variability across seeds. To facilitate reproducibility, the source code is publicly available at <https://github.com/funny-rl/SPECTra>

TABLE 3. Hyperparameter settings of SMACv2 and GRF experiments.

Hyperparameter	SMACv2 (QMIX)			SMACv2 (VDN)			GRF			
	SPECTra QMIX ⁺	HPN QMIX	UPDeT QMIX	SPECTra VDN	HPN VDN	UPDeT VDN	SPECTra QMIX ⁺	HPN QMIX	SPECTra VDN	HPN VDN
mac	ss_mac	hpn_mac	updet_mac	ss_mac	hpn_mac	updet_mac	ss_mac	hpn_mac	ss_mac	hpn_mac
agent	ss_rnn	hpn_rnn	updet_agent	ss_rnn	hpn_rnn	updet_agent	ss_rnn	hpn_rnn	ss_rnn	hpn_rnn
mixer	ss_mixer	qmix	qmix	vdn	vdn	vdn	ss_mixer	qmix	vdn	vdn
hidden_size	64	64	32	64	64	32	64	64	64	64
n_head	4	–	3	4	–	3	4	–	4	–
depth	1	–	2	1	–	2	1	–	1	–
mix_n_head	4	–	–	–	–	–	4	–	–	–
mix_embed	#agents	32	32	–	–	–	–	32	–	–
hyper_embed	32	64	64	–	–	–	32	64	–	–
batch_size	128	128	128	128	128	128	32	32	32	32

Hyperparameters in SMACv2 Experiments: Table 3 summarizes the hyperparameter settings for all experiments. In SMACv2, the batch_size is set differently depending on the total number of entities to manage memory stably. Specifically, 128 for 5 vs 5 environments, 32 for 10vs10, and 10 for 20vs20. In addition, SPECTra-QMIX⁺ uses the attention scores of ST-HyperNet as the weight and bias of the mixing network, and these values change dynamically with the number of agents. Therefore, the number of agents dynamically determines the mixing_embed_dim.

Hyperparameters in GRF Experiments: GRF is one of the most challenging MARL benchmark environments due to its score-based sparse rewards and long-horizon episodes. Using only the score-based reward function leads to over-exploration, which significantly degrades training speed. Therefore, it is necessary to customize the reward function to mitigate over-exploration. Moreover, since the action space varies across environments, each model should use an optimized action sampling method tailored to the permutation problem in each setting. This makes GRF essential for evaluating the generalized performance of the SPECTra framework. Since the GRF implementation of HPN [18] is not publicly available, we designed our GRF training pipeline based on the GitHub project published by CDS [48] and implemented a model following the architecture described in the HPN paper. We also accelerated training by using additional action masks and reward functions. Table 3 provides the hyperparameter settings for the GRF experiments.

Experimental Environment: Experiments were conducted on the Intel(R) Xeon(R) Gold 6240R CPU @ 2.40GHz 24-core processor with 768GB RAM and NVIDIA A10 D6 24GB.

A. COMPARISON WITH BASELINES

To evaluate the performance of our proposed algorithm, we first compare the performance of HPN, UPDeT, and SPECTra based on VDN and QMIX algorithms. The results of the experiments in the 10 vs 10 scenarios of SMACv2 are shown in Figure 3.

The results show that SPECTra-QMIX⁺ and SPECTra-VDN achieve strong performance compared to the corresponding QMIX and VDN-based models. SPECTra-QMIX

and SPECTra-VDN train much faster than UPDeT-QMIX and UPDeT-VDN, which utilize self-attention, due to the low computational cost of SAQA. Moreover, as discussed in Section IV-C, the difference in computation time between QMIX and VDN-based algorithms is not significant, indicating that the computational cost of the agent network has a greater impact on training time than that of the mixing network.

Figure 4 compares the inference time of each algorithm with 1,000 samples and reveals that SAQA requires significantly less computation time than self-attention, supporting the results in Figure 3. SPECTra is faster than UPDeT and outperforms HPN, and we confirm that this trend is consistent as the number of agents increases.

It is important to note that SPECTra-QMIX⁺ achieves superior performance with significantly fewer parameters than the other baselines. Unlike the baselines, where the number of parameters increases with the number of agents, SPECTra-QMIX⁺ and SPECTra-VDN maintain a fixed parameter count regardless of the number of agents. The number of parameters in SPECTra-QMIX⁺ is about 50% of UPDeT-QMIX and 33% of HPN-QMIX, and this gap widens further as the number of agents increases. SPECTra-VDN also has only about 48% of the parameters of HPN-VDN, yet it performs competitively or superiorly in most environments. UPDeT-VDN has fewer parameters than SPECTra-VDN, but its scalability is limited due to the sharp rise in computational cost. Meanwhile, despite having more parameters than SPECTra-QMIX⁺, UPDeT-QMIX does not achieve better performance.

Figures 5 and 6 compare SPECTra and other baseline algorithms across all SMACv2 scenarios. SPECTra-QMIX⁺ shows outstanding performance in overall experiments.

B. CURRICULUM LEARNING

As shown in Table 1, SPECTra-QMIX⁺ is highly scalable, allowing it to adapt to larger environments without requiring additional parameters. This scalability makes it particularly well-suited for curriculum learning in settings with dynamically changing agent populations. While previous models like UPDeT have significantly improved transferability at the

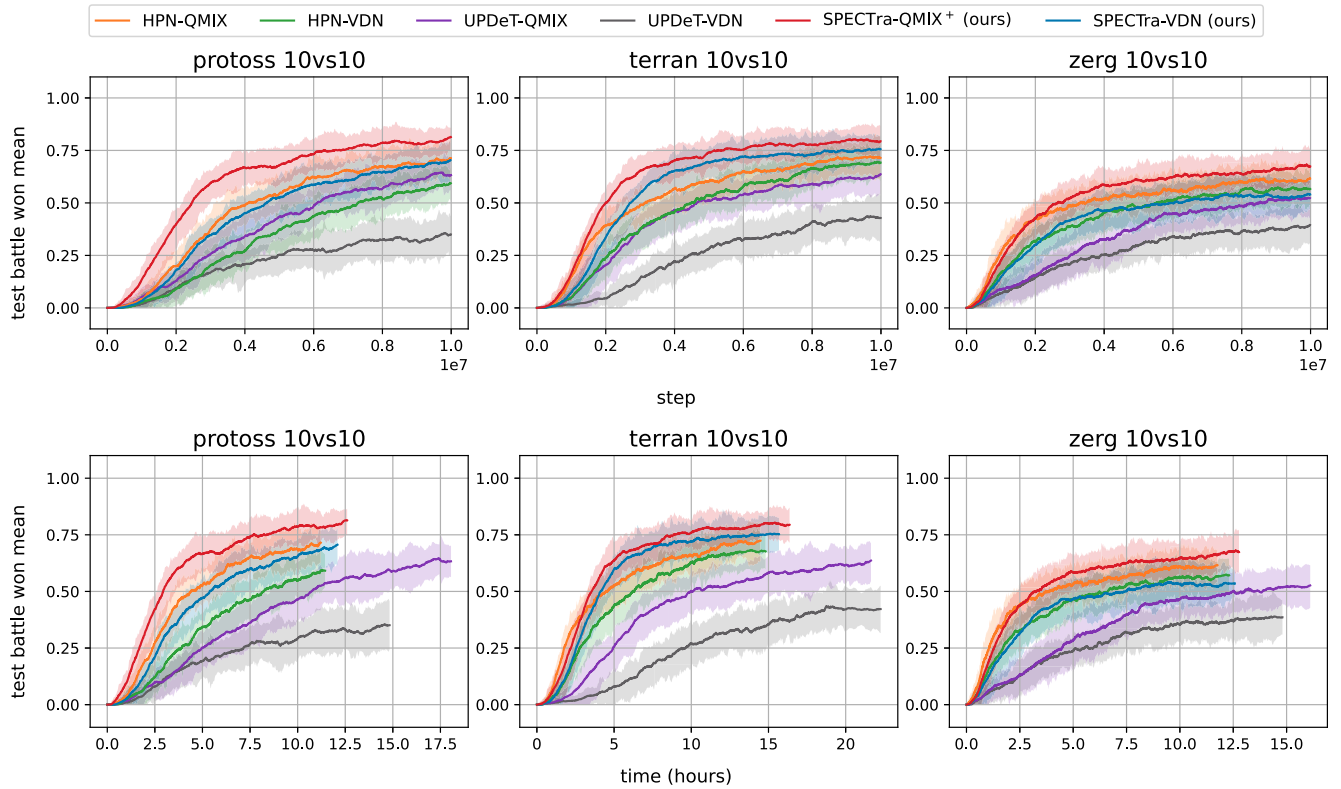


FIGURE 3. Comparison of SPECTra-QMIX⁺ and SPECTra-VDN against baselines in SMACv2. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.



FIGURE 4. Comparison of inference time between the proposed SPECTra and baseline methods.

agent network level, they have not been extended to non-linear mixing networks.

Specifically, 30% of the total training occurs in a 5 vs 5 environment before transitioning to the target evaluation environment for the remaining 70%. We denote the variant of our model for curriculum learning by SPECTra-QMIX⁺CL. As shown in Figure 7, SPECTra-QMIX⁺CL achieves superior performance compared to the standard SPECTra-QMIX⁺,

demonstrating that even a simple curriculum learning strategy can enhance transferability.

Moreover, to evaluate the transfer ability of SPECTra-VDN and SPECTra-QMIX⁺, we perform transfer learning from 5 vs 5 to 10 vs 10 to assess how well SPECTra, HPN, and UPDeT generalize learned policies. To verify that the models learn generalizable strategies rather than unit-specific repeated actions, we conduct both homogeneous and heterogeneous transfer learning. The results in Figure 8 demonstrate that SPECTra-VDN and SPECTra-QMIX⁺ perform well in both settings, confirming that they can fully utilize pre-trained models and justifying the curriculum learning results in Figure 7.

In the SMACv2 20 vs 20 environments, SPECTra-QMIX⁺ performs well despite having only 24% of the parameters of HPN-QMIX. On the other hand, UPDeT-QMIX, which uses self-attention, shows a sharp drop in learning performance as the number of entities increases, despite having 3.4 times more parameters than SPECTra-QMIX⁺. This suggests that self-attention-based computation cannot cope with the exponential growth of complexity as the number of entities increases.

C. GRF EXPERIMENTS

To verify the generalizability of our algorithm, we conduct comparative experiments not only in SMACv2 but also in GRF. Unlike SMACv2, GRF controls all agents of the

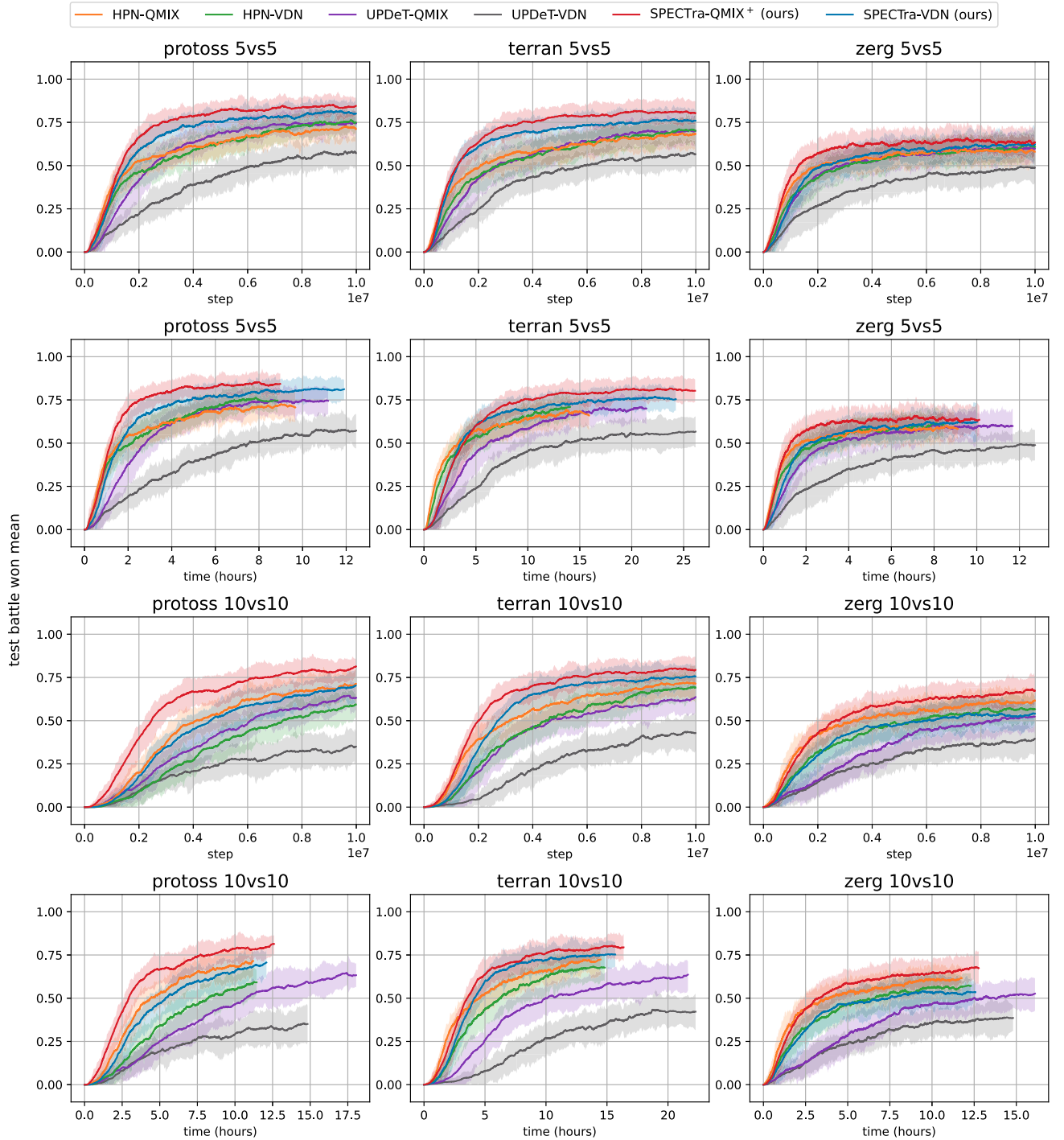


FIGURE 5. Overall comparison over SMACv2 challenges 5 vs 5 and 10 vs 10. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

home team except the goalkeeper and provides sparse team rewards based solely on goal scoring. Moreover, addressing the permutation problem is crucial in GRF, as the vast action space leads to an enormous state space. We use HPN as the baseline, as it has demonstrated strong performance in this environment.

Unlike SMACv2, GRF has a fixed action space, which requires different methods to solve the permutation problem. This difference affects the direct applicability of the proposed policy decoupling module, because policy decoupling assumes that actions can be separated into self-directed actions and entity-interaction actions whose outputs should

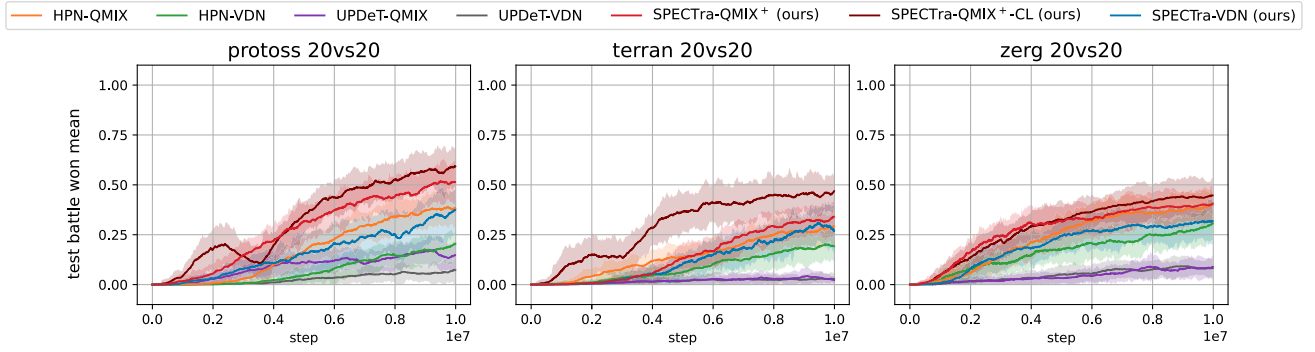


FIGURE 6. Overall comparison over SMACv2 challenges 20 vs 20. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

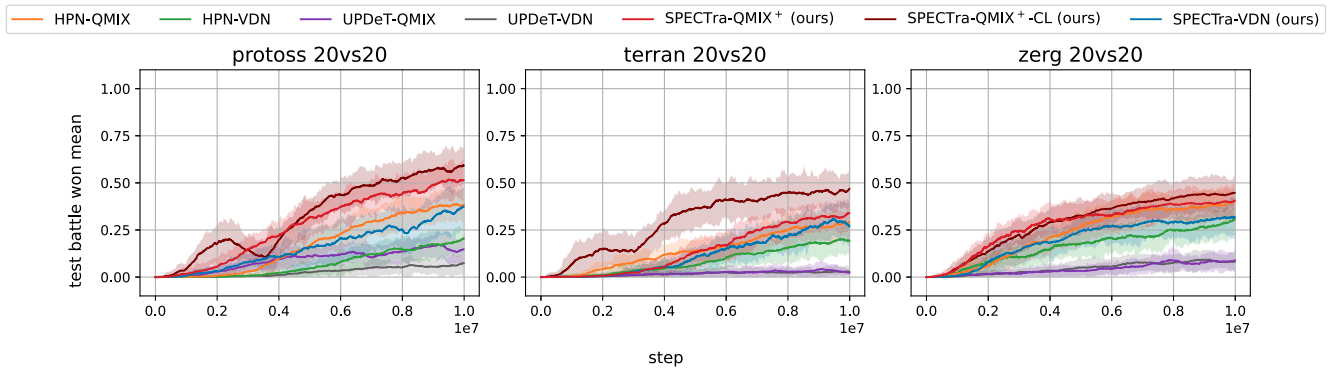


FIGURE 7. Comparison of SPECTra-QMIX⁺ and SPECTra-VDN with curriculum learning against baselines in SMACv2. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

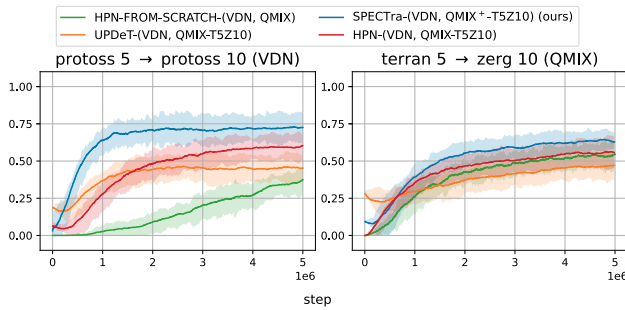


FIGURE 8. Comparison of SPECTra-QMIX⁺ and SPECTra-VDN with transfer learning against baselines in Protoss 5 → Protoss 10 and Terran 5 → Zerg 10. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

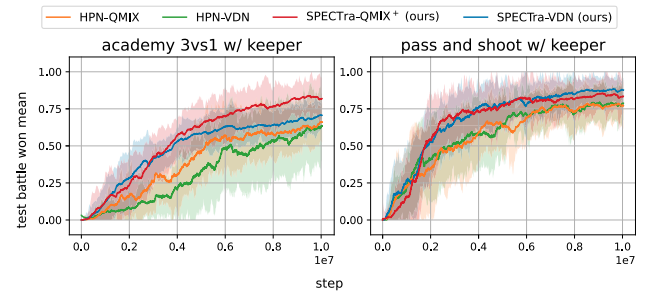


FIGURE 9. Comparison of SPECTra-QMIX⁺ and SPECTra-VDN against baselines in GRF. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

follow entity permutations. In GRF, however, several actions, such as pass-related actions, are defined within a fixed global action space rather than as explicitly entity-indexed interaction actions. Therefore, we adapt SPECTra to GRF by preserving the scalable agent representation and value-mixing components, while replacing the entity-wise policy decoupling for pass actions with mean pooling. Since HPN does not currently provide an available implementation for GRF, we reproduce the algorithm as faithfully as possible

based on the details available [18]. We note that the GRF comparison should be interpreted with this implementation detail in mind, because the HPN baseline in GRF is based on our reimplementation rather than an official released implementation. To make the comparison as fair as possible, we applied the same GRF training pipeline, reward design, action masks, training budget, evaluation intervals, and random seed settings to all compared methods. Therefore, the GRF experiments are intended to provide additional evidence of generalization beyond SMACv2, while

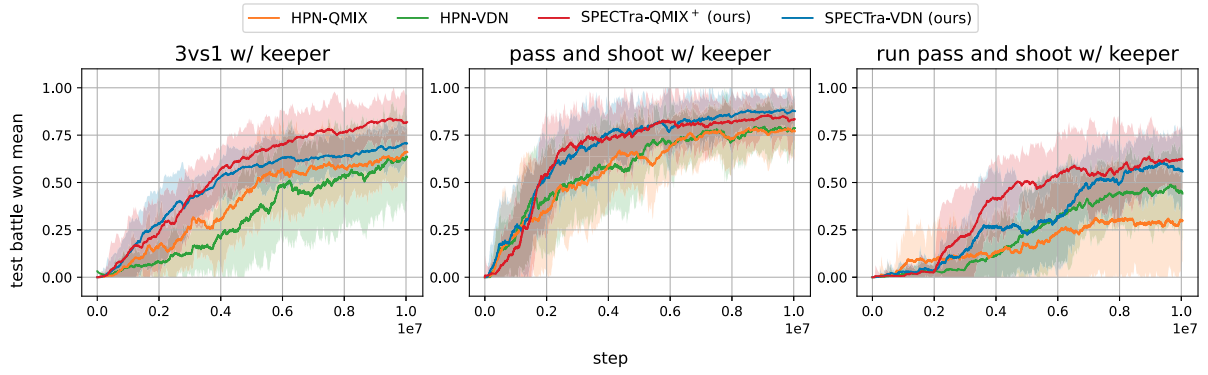


FIGURE 10. Comparison of SPECTra-QMIX⁺ and SPECTra-VDN against baselines on three GRF maps. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

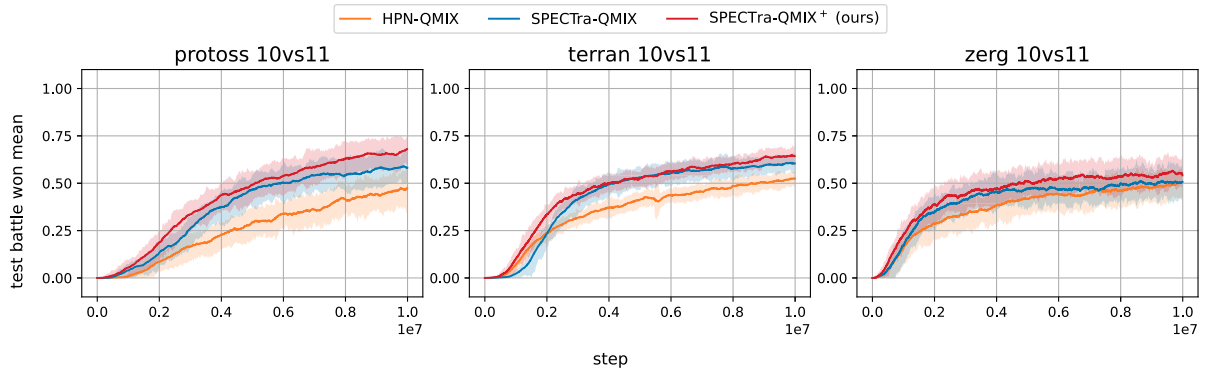


FIGURE 11. Ablation experiments for SAQA and ST-HyperNet modules. Results are averaged over 10 random seeds.

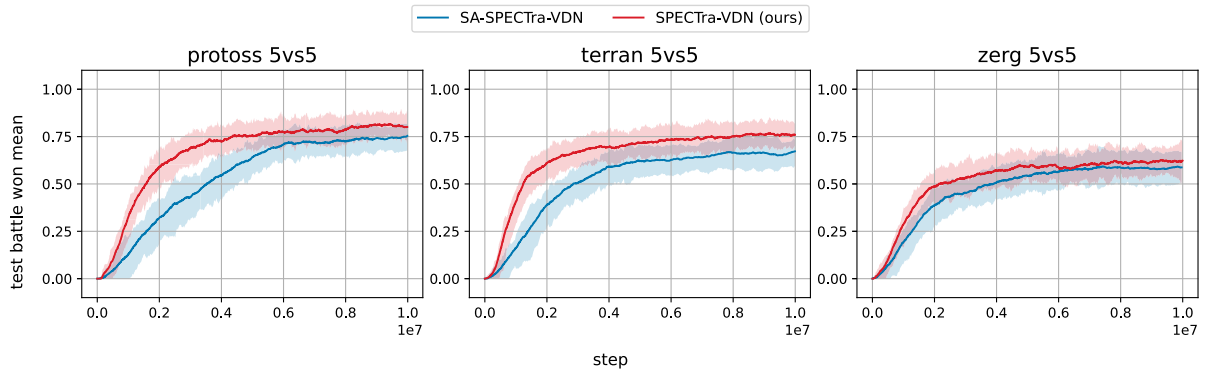


FIGURE 12. Comparison of SAQA and self-attention layers with mean pooling across three races. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

SMACv2 remains our primary benchmark for controlled comparison. Accordingly, to make SPECTra-QMIX⁺ and SPECTra-VDN applicable to GRF, we do not use policy decoupling for the fixed pass-action structure and instead apply mean pooling across agents for pass actions. This adaptation is specific to the GRF action-space design and does not affect the SMACv2 experiments, where the full SPECTra architecture is evaluated under entity-interaction

action settings. The experimental results in Figure 9 demonstrate that both SPECTra-QMIX⁺ and SPECTra-VDN achieve strong learning performance in GRF, further confirming that SPECTra exhibits robust and effective performance across various MARL environments.

Figure 10 shows the comparison between SPECTra and HPN. Each is combined with QMIX or VDN. The scenarios used in the figure are *academy 3 vs 1 with keeper*,

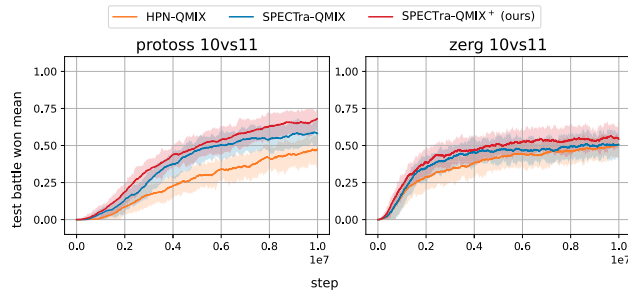


FIGURE 13. Ablation experiments for the SAQA and ST-HyperNet modules. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

academy pass and shoot with keeper, and *academy run pass and shoot with keeper*, respectively.

D. ABLATION STUDY

We conduct ablation studies to evaluate the contribution of each proposed component in the SMACv2 environment. Figure 11 confirms that the proposed agent network and mixing network both contribute to performance improvement. Additionally, Figure 12 shows that SAQA is more effective than conventional self-attention for embedding generation and directly impacts performance. We use these ablations to isolate the main components of SPECTra without introducing additional unrelated variants. The comparison between SPECTra-QMIX and HPN-QMIX evaluates the effect of the proposed agent network, while the comparison between SPECTra-QMIX⁺ and SPECTra-QMIX evaluates the contribution of ST-HyperNet and the permutation-free non-linear mixing network. The comparison between SPECTra-VDN and SA-SPECTra-VDN further examines the impact of replacing SAQA with conventional self-attention. In addition, the curriculum and transfer learning experiments in Section V-B analyze the effect of curriculum learning and transferability under varying team sizes. Together, these ablations are intended to validate the three main design decisions of SPECTra: the SAQA-based agent representation, ST-HyperNet-based permutation-free mixing network, and the use of curriculum/transfer learning under varying team sizes.

First, we analyze the effectiveness of the proposed agent network and mixing network. We conduct an ablation study in a 10 vs 11 scenario across all races in SMACv2 (Protoss, Terran, and Zerg) using SPECTra-QMIX⁺, HPN-QMIX, and SPECTra-QMIX, where SPECTra-QMIX replaces the mixer of HPN-QMIX with ST-HyperNet. The results are presented in Figure 13.

By comparing the experimental results of SPECTra-QMIX and HPN-QMIX, we confirm that our agent networks outperform the agent networks in the original HPN model. This supports our claim that performing attention on each entity's observation embeddings is more effective than mean pooling. Furthermore, the comparison between SPECTra-QMIX⁺ and

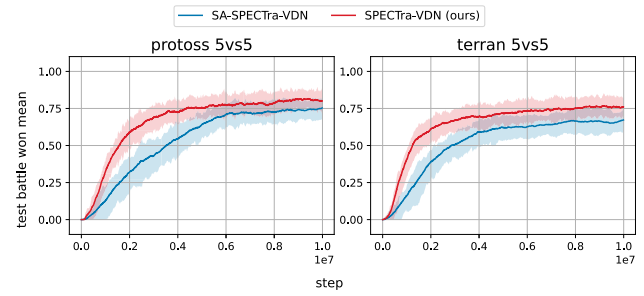


FIGURE 14. Comparison of SAQA and self-attention layers with mean pooling. Results are averaged over 10 random seeds, and shaded regions indicate variability across seeds.

SPECTra-QMIX demonstrates that the permutation-invariance property of our non-linear mixing network significantly contributes to improving model performance.

1) SAQA vs SELF-ATTENTION

Recall that we propose SAQA to address the high computational cost and sample complexity of conventional self-attention. To evaluate the architectural differences, we conduct experiments in a 5 vs 5 scenario across all races using SPECTra-VDN and SA-SPECTra-VDN, where SA-SPECTra-VDN is derived from SPECTra-VDN by replacing SAQA with a self-attention layer. Figure 14 shows the results.

Note that we use all observations along with the current agent information as queries, and the output embeddings of the self-attention are aggregated using a mean pooling operation. The experimental results demonstrate that self-attention unnecessarily increases sample complexity, whereas SAQA achieves higher performance with significantly lower computational cost.

We also conduct a qualitative analysis to examine how SAQA and self-attention layers interpret input observations differently. Figure 15 compares the attention scores of SAQA and self-attention when an observation from SMACv2 is input into SPECTra and UPDeT. To analyze the response to unobserved entities processed as zero vectors, we trained the models without applying attention masks. While SPECTra selected the attack action, UPDeT chose the move action. In the given state, the optimal action is to attack and eliminate Enemy 4 as quickly as possible, since it has significantly lower HP compared to Enemy 1. SAQA utilizes residual connections, allowing it to pay less attention to the query's own information, enabling each head to focus on different aspects of the four observed entities. This indicates that the model can learn a unidirectional relationship centered on the observer. In contrast, UPDeT uses each entity's vector as the basis for selecting an action corresponding to that entity. The most crucial information for action selection should be the relationship between the observer agent and the target entity. To compute the action value for Enemy 4, the model should assign a high attention score to the relationship between the agent and Enemy 4. However, UPDeT tends to focus more

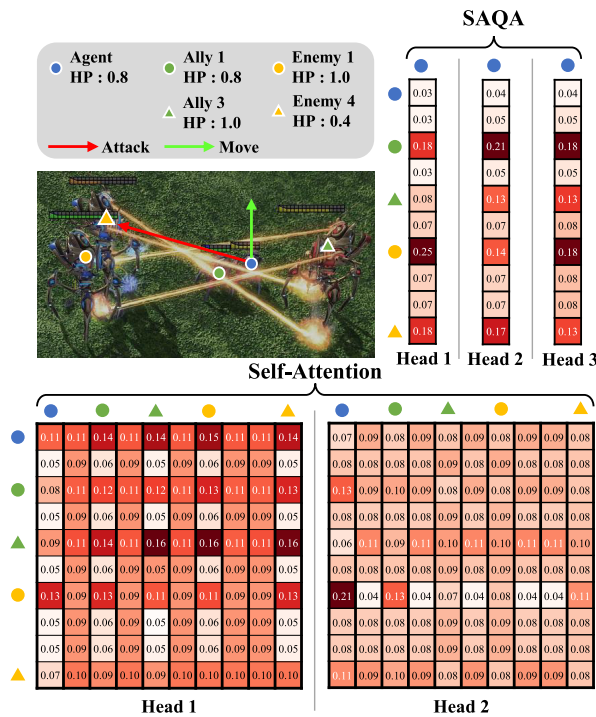


FIGURE 15. Qualitative example of attention maps computed from SAQA and self-attention layers (UPDeT) in the SMACv2 environment.

on Ally 3 than on Enemy 4, suggesting that it struggles to prioritize the relevant pairwise relationships among entities.

VI. LIMITATION

Although SPECTra improves scalability and permutation-freeness in entity-based cooperative MARL, it has several limitations. First, the proposed permutation-free formulation assumes that observations can be decomposed into entity-wise features and that actions can be separated into self-directed or entity-interaction actions. Therefore, it may not directly apply to environments with non-decomposable observations, non-exchangeable entity identities, or globally ordered action structures. Second, while SAQA reduces attention complexity from $O(n^2d)$ to $O(nd)$, it focuses on observer-centered interactions and may be less suitable when dense pairwise interactions among all entities are essential.

VII. CONCLUSION

In this work, we have proposed a MARL framework called SPECTra that leverages a Transformer-based neural architecture to address the scalability and permutation problems. By utilizing SAQA, we mitigate the high computational cost of self-attention layers while maintaining the ability to learn complex interactions among many entities. Our experimental results confirm that SAQA accelerates the training process and achieves superior performance compared to baseline approaches. Additionally, we successfully address the scalability and permutation problems in non-linear mixing networks by introducing the Set Transformer-inspired hypernetwork called ST-HyperNet.

Furthermore, the SPECTra framework demonstrates strong applicability and generality through transfer learning and curriculum learning experiments. Our study presents an efficient way to adopt attention mechanisms in highly complex MARL scenarios and holds the potential to be extended to real-world-scale environments that require modeling interactions among a large number of agents.

ACKNOWLEDGMENT

The authors acknowledge the Urban Big Data and AI Institute of the University of Seoul supercomputing resources (<http://ubai.uos.ac.kr>) made available for conducting the research reported in this paper. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Science and ICT (MSIT) (No. RS-2024-00456065).

(Hyunwoo Park and Baekryun Seong contributed equally to this work.)

REFERENCES

- [1] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.
- [2] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016.
- [3] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, Nov. 2019.
- [4] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Canberra, ACT, Australia, Dec. 2020, pp. 737–744.
- [5] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, Sep. 2013.
- [6] P. F. Christiano, J. Leike, T. B. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4299–4307.
- [7] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. Adv. Neural Inf. Process. Syst. 36*, New Orleans, LA, USA, Dec. 2023, pp. 53728–53741.
- [8] D. Guo et al., "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," 2025, *arXiv:2501.12948*.
- [9] D. T. Nguyen, A. Kumar, and H. C. Lau, "Credit assignment for collective multiagent RL with global rewards," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, vol. 31, 2018, pp. 8113–8124.
- [10] A. L. C. Bazzan, "Opportunities for multiagent systems and multiagent reinforcement learning in traffic control," *Auto. Agents Multi-Agent Syst.*, vol. 18, no. 3, pp. 342–375, Jun. 2009.
- [11] F. Lin, S. Huang, T. Pearce, W. Chen, and W.-W. Tu, "TiZero: Mastering multi-agent football with curriculum learning and self-play," 2023, *arXiv:2302.07515*.
- [12] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," 2018, *arXiv:1803.11485*.
- [13] W. Wang, T. Yang, Y. Liu, J. Hao, X. Hao, Y. Hu, Y. Chen, C. Fan, and Y. Gao, "From few to more: Large-scale dynamic multiagent curriculum learning," in *Proc. 34th AAAI Conf. Artif. Intell.*, New York, NY, USA, 2019, pp. 7293–7300.
- [14] P. Li, T. Hongyao, Y. Tianpei, X. Hao, T. Sang, Z. Yan, J. Hao, M. E. Taylor, T. Wenyuan, W. Zhen, and B. Fazl, "PMIC: Improving multi-agent reinforcement learning with progressive mutual information collaboration," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2022, pp. 12979–12997.

- [15] M. Wen, J. G. Kuba, R. Lin, W. Zhang, Y. Wen, J. Wang, and Y. Yang, "Multi-agent reinforcement learning is a sequence modeling problem," 2022, *arXiv:2205.14953*.
- [16] I.-J. Liu, R. A. Yeh, and A. G. Schwing, "PIC: Permutation invariant critic for multi-agent deep reinforcement learning," in *Proc. 3rd Annu. Conf. Robot Learn.*, Oct. 2019, pp. 590–602.
- [17] S. Hu, F. Zhu, X. Chang, and X. Liang, "UPDeT: Universal multi-agent RL via policy decoupling with transformers," in *Proc. 9th Int. Conf. Learn. Represent.*, Virtual Event, Austria, 2021, pp. 1–15.
- [18] J. Hao, X. Hao, H. Mao, W. Wang, Y. Yang, D. Li, Y. Zheng, and Z. Wang, "Boosting multiagent reinforcement learning via permutation invariant and permutation equivariant networks," in *Proc. 11th Int. Conf. Learn. Represent.*, Kigali, Rwanda, 2023, pp. 1–24.
- [19] C. Sun, M. Shen, and J. P. How, "Scaling up multiagent reinforcement learning for robotic systems: Learn an adaptive sparse communication graph," 2020, *arXiv:2003.01040*.
- [20] D. Ha, A. Dai, and Q. V. Le, "HyperNetworks," 2016, *arXiv:1609.09106*.
- [21] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," 2017, *arXiv:1706.02275*.
- [22] C. Amato, "An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning," 2024, *arXiv:2409.03052*.
- [23] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, "Value-decomposition networks for cooperative multi-agent learning," 2017, *arXiv:1706.05296*.
- [24] K. Son, D. Kim, W. J. Kang, D. E. Hostallero, and Y. Yi, "QTRAN: Learning to factorize with transformation for cooperative multi-agent reinforcement learning," 2019, *arXiv:1905.05408*.
- [25] J. Hu, S. Jiang, S. A. Harding, H. Wu, and S.-W. Liao, "Rethinking the implementation tricks and monotonicity constraint in cooperative multi-agent reinforcement learning," 2021, *arXiv:2102.03479*.
- [26] J. Wang, Z. Ren, T. Z. Liu, Y. Yu, and C. Zhang, "QPLEX: Duplex dueling multi-agent Q-learning," in *Proc. 9th Int. Conf. Learn. Represent.*, Virtual Event, Austria, 2021, pp. 1–27.
- [27] Y. Yang, J. Hao, B. Liao, K. Shao, G. Chen, W. Liu, and H. Tang, "Qatten: A general framework for cooperative multiagent reinforcement learning," 2020, *arXiv:2002.03939*.
- [28] T. Rashid, G. Farquhar, B. Peng, and S. Whiteson, "Weighted QMIX: Expanding monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 10199–10210.
- [29] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson, "The StarCraft multi-agent challenge," in *Proc. Int. Joint Conf. Auto. Agents Multiagent Syst.*, Montreal, QC, Canada, May 2019, pp. 2186–2188.
- [30] S. Hazra, P. Dasgupta, and S. Dey, "Addressing permutation challenges in multi-agent reinforcement learning," in *Proc. Int. Joint Conf. Auto. Agents Multiagent Syst.*, May 2024, pp. 2303–2305.
- [31] J. McClellan, N. Haghani, J. Winder, F. Huang, and P. Tokekar, "Boosting sample efficiency and generalization in multi-agent reinforcement learning via equivariance," 2024, *arXiv:2410.02581*.
- [32] Y. Li, L. Wang, J. Yang, E. Wang, Z. Wang, T. Zhao, and H. Zha, "Permutation invariant policy optimization for mean-field multi-agent reinforcement learning: A principled approach," 2021, *arXiv:2105.08268*.
- [33] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, "Deep sets," 2017, *arXiv:1703.06114*.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [35] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2008.
- [36] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, Toulon, France, 2017, pp. 1–14.
- [37] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and P. S. Yu, "A comprehensive survey on graph neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [38] M. Zhou, Y. Chen, Y. Wen, Y. Yang, Y. Su, W. Zhang, D. Zhang, and J. Wang, "Factorized Q-learning for large-scale multi-agent systems," in *Proc. 1st Int. Conf. Distrib. Artif. Intell.*, Beijing, China, Oct. 2019, pp. 1–7.
- [39] A. Tampuu, T. Matiisen, D. Kodelja, I. Kuzovkin, K. Korjus, J. Aru, J. Aru, and R. Vicente, "Multiagent cooperation and competition with deep reinforcement learning," 2015, *arXiv:1511.08779*.
- [40] R. Wang, L. Zheng, W. Qiu, B. He, B. An, Z. Rabinovich, Y. Hu, Y. Chen, T. Lv, and C. Fan, "Towards skilled population curriculum for multi-agent reinforcement learning," 2023, *arXiv:2302.03429*.
- [41] Q. Long, Z. Zhou, G. Abhibav, F. Fang, Y. Wu, and X. Wang, "Evolutionary population curriculum for scaling multi-agent reinforcement learning," in *Proc. 8th Int. Conf. Learn. Represent.*, Addis Ababa, Ethiopia, Apr. 2020, pp. 1–18.
- [42] J. Chen, Y. Zhang, Y. Xu, H. Ma, H. Yang, J. Song, Y. Wang, and Y. Wu, "Variational automatic curriculum learning for sparse-reward cooperative multi-agent problems," in *Proc. Adv. Neural Inf. Process. Syst. 34: Annu. Conf. Neural Inf. Process. Syst.*, 2021, pp. 9681–9693.
- [43] D. S. Bernstein, R. Givan, N. Immerman, and S. Zilberstein, "The complexity of decentralized control of Markov decision processes," *Math. Operations Res.*, vol. 27, no. 4, pp. 819–840, Nov. 2002.
- [44] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs*. Cham, Switzerland: Springer, 2016.
- [45] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proc. 36th Int. Conf. Mach. Learn.*, Long Beach, California, Jun. 2018, pp. 3744–3753.
- [46] S. Zhuoran, Z. Mingyuan, Z. Haiyu, Y. Shuai, and L. Hongsheng, "Efficient attention: Attention with linear complexities," in *Proc. IEEE Winter Conf. Appl. Comput. Vis. (WACV)*, Waikoloa, HI, USA, Jan. 2021, pp. 3530–3538.
- [47] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, and J. Hoffman, "Hydra attention: Efficient attention with many heads," in *Proc. Comput. Vis.-ECCV Workshops*, Oct. 2022, pp. 35–49.
- [48] C. Li, T. Wang, C. Wu, Q. Zhao, J. Yang, and C. Zhang, "Celebrating diversity in shared multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst., Annu. Conf. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 3991–4002.



HYUNWOO PARK is currently pursuing the B.S. degree in artificial intelligence with the University of Seoul. His research interests include multi-agent reinforcement learning, deep learning, and cooperative AI.



BAEKRYUN SEONG received the B.S. degree in artificial intelligence from the University of Seoul, Seoul, Republic of Korea, where he is currently pursuing the M.S. degree in artificial intelligence. His research interests include spiking neural networks, multi-agent reinforcement learning, and verification of deep learning models.



SANG-KI KO received the Ph.D. degree in computer science from Yonsei University, Seoul, Republic of Korea. He is currently an Associate Professor with the Department of Artificial Intelligence, University of Seoul, Seoul. His research interests include theoretical computer science, machine learning, and multi-agent reinforcement learning.

...